

COMMUNAUTÉ FRANÇAISE
HAUTE ÉCOLE ROBERT SCHUMAN
DÉPARTEMENT ÉCONOMIQUE
6800 LIBRAMONT

Implémentations des l'interfaces Deque et BigPriorityQueue

Travail présenté pour le cours de
Paradigme orienté objet 2
Bachelier en Informatique

ANDRÉ Allan
DECEMBRE 2025

Je soussigné·e, Andre Allan, déclare sur l'honneur que cette production (travail / TFE mémoire / rapport de stage /.....)[1] est entièrement personnelle, fruit de ma réflexion, de mon travail et de mon écriture.

Je certifie que toutes les idées que j'ai pu avancer qui proviennent d'ouvrages, d'articles, de mémoires et/ou de sources sont référencées selon les normes en vigueur. Toutes les citations textuelles sont en outre signalées dans le respect des normes typographiques de référencement.

Je certifie également que :

- ☐ je n'ai en aucune manière utilisé d'IA



[2]

Table des matières

Introduction 3

1. Choix d'implémentation 4

2. Tests 5

a. DequeInterface 5

b. Question4 5

c. BigPriorityQueueInterface 5

3. Complexité temporelle 6

a. DequeInterface 6

b. BigPriorityQueueInterace 15

Conclusion 16

Annexes 17

Introduction

Il existe de nombreux moyen de stocker de grandes quantités de données. Mais quelle est la plus efficace ? C'est ce que j'ai essayé de découvrir en créant 3 listes. La première utilise un tableau de manière naïve, la deuxième un tableau de manière plus optimisée, et la dernière est une liste chaînée.

Il est possible, pour chacune des listes, d'ajouter ou de retirer des éléments au début ou à la fin, mais jamais au milieu. De plus, on peut faire des recherches dans ces listes, que ce soient des éléments en particulier ou des éléments répondant à des critères voulus.

Avant de vous présenter mes conclusions, je vais vous faire part de mes choix d'implémentations. Je vous montrerais ensuite les tests effectués sur ces trois listes. Enfin, je comparerais l'efficacité temporelle de ces listes pour pouvoir découvrir laquelle est la plus optimale.

1. Choix d'implémentation

Pour ArrayDequeNaive, j'ai fait en sorte de réduire au maximum l'espace mémoire utilisé pour les données en créant un tableau de la taille exacte du nombre de données présentes. Le programme agrandit le tableau à chaque fois qu'un élément est ajouté et le réduit à chaque fois qu'un élément est retiré. Il n'y a donc jamais de case vide dans le tableau, ce qui réduit l'espace mémoire utilisé pour sauvegarder les données, mais en contrepartie réduit énormément l'efficacité puisque le programme créera un nouveau tableau à chaque ajout et retrait.

Dans ArrayDeque, j'ai fait un constructeur prenant en paramètre la taille du tableau voulu par l'utilisateur, car s'il connaît la taille du tableau qui lui est nécessaire, cela permet de créer un tableau avec la taille idéale pour lui et peut permettre d'éviter d'agrandir le tableau plus tard. De plus, lors des tests, cela me permet de déclarer un petit tableau et d'ainsi vérifier que le tableau est bien agrandi sans devoir ajouter de nombreuses données. Le désavantage de ce système, c'est que l'utilisateur peut créer un tableau d'une centaine de cases ou plus, mais n'en utiliser que quelques-unes, causant un gaspillage mémoire conséquent…

ArrayDeque utilise un tableau circulaire pour éviter le gaspillage mémoire et optimiser l'efficacité. En effet, le programme ajoutera les données au centre du tableau et, si un des bords est atteint, continuera de l'autre côté. Avec ce système, il n'est pas rare que la donnée placée à la tête de la liste se trouve en réalité à un emplacement situé après la donnée placée à la fin de la liste. De plus, cela permet de n'agrandir le tableau que lorsque le nombre d'éléments présents est égal à la taille du tableau.

La classe Personne me permet de tester efficacement l'interface BigPriorityQueueInterface, car j'avais besoin d'une classe possédant deux variables pour tester que le tri s'effectuait correctement avec deux comparator différents.

En faisant les tests de BigPriorityQueueInterface, je me suis rendu compte que la fonction contains ne fonctionnait pas correctement, car elle utilisait la fonction equals de la classe Object et pas celle de Personne. En testant dans DequeInterface, j'ai découvert le même problème. Pour remédier à cela, la seule solution que j'ai trouvée a été d'overrider la fonction equals(Object obj) dans la classe Personne.

2. Tests

a. DequeInterface

Les tests pour ArrayDequeNaive, ArrayDeque et LinkedDeque se trouvent dans le fichier TestDequeInterface.java. Pour tester les classes, il faut modifier la déclaration de variable au début de chaque fonction. J'avais essayé de faire, dans le fichier de test, la fonction :

```
@BeforeEach
private void init() {
        deque = new LinkedDeque<String>();
}
```

En déclarant deque comme variable globale de type DequeInterface, ce qui aurait permis de tester l'entièreté des classes en ne changeant qu'une seule fois la déclaration, mais lorsque je lançais les tests le programme m'indiquait que deque était null.
Je ne savais pas comment tester la fonction forEach() efficacement, alors j'ai fait un affichage, et c'est au programmeur de vérifier les résultats obtenus.

ArrayDequeNaive, ArrayDeque et LinkedDeque passent les tests, mais il est nécessaire que le type utilisé override la méthode equals(Object obj), car autrement, certaines méthodes fonctionnent mal, notamment contains qui va utiliser la méthode equals de Object, ce qui renvoie true uniquement si les adresses mémoires des deux objets sont les mêmes.

b. Question4

Les tests se trouvent dans le fichier Question4.java lui-même, en dessous de chaque fonction correspondante. Comme pour les tests de DequeInterface, je n'ai pas réussi à créer de fonction de déclaration.

ArrayDequeNaive, ArrayDeque et LinkedDeque passent les tests.

c. BigPriorityQueueInterface

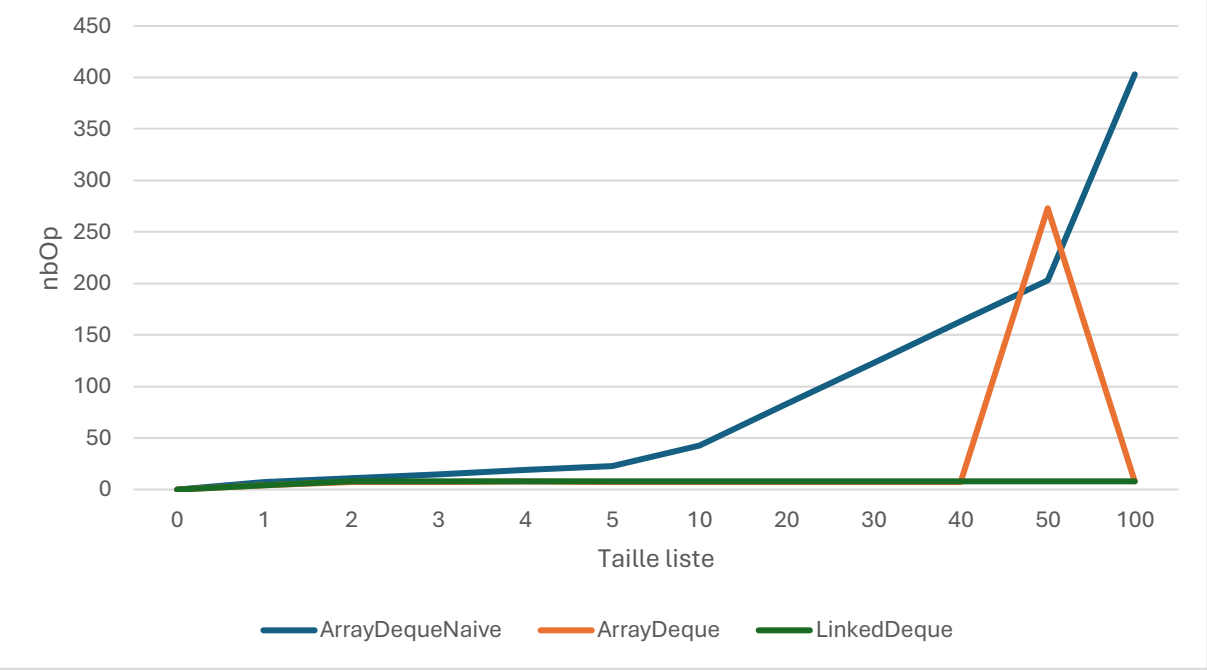
Les tests se trouvent dans le fichier TestBigPriorityQueue.java. Comme précédemment, je n'ai pas réussi à créer de fonction de déclaration.

ArrayBigPriorityQueueNaive, ArrayBigPriorityQueue et LinkedBigPriorityQueue passent les tests, mais il est nécessaire que le type utilisé override la méthode equals(Object obj), car autrement, certaines méthodes fonctionnent mal, notamment contains qui va utiliser la méthode equals de Object, ce qui renvoie true uniquement si les adresses mémoires des deux objets sont les mêmes.

3. Complexité temporelle

a. DequeInterface

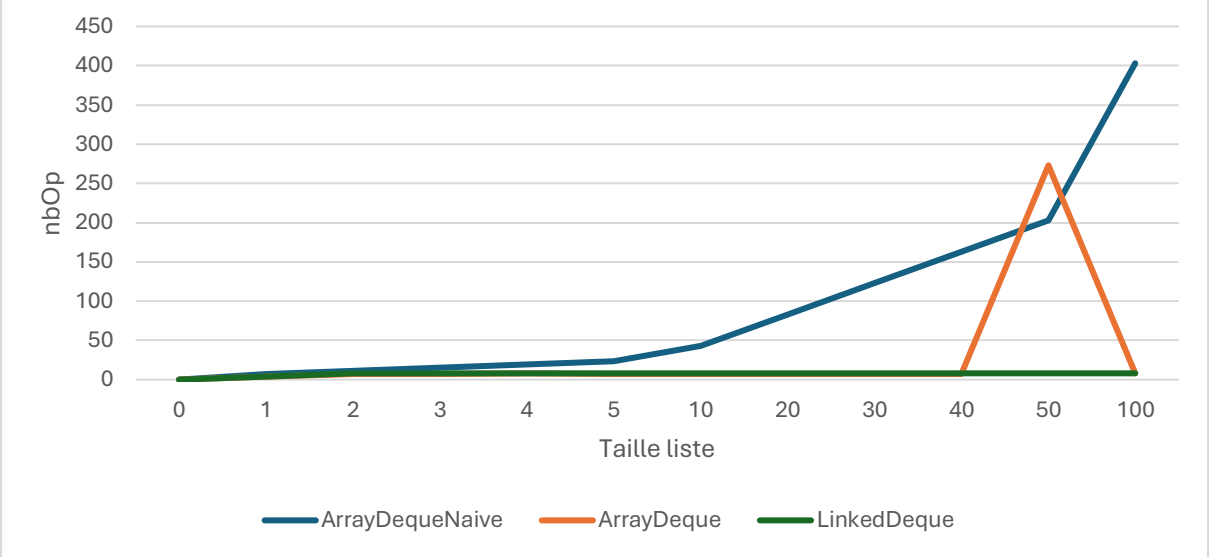
offerFirst(E e)	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(1)



On peut constater une augmentation élevée du nombre d'opérations dans le ArrayDeque. Cela est dû au fait que l'on ajoute un 50e élément dans un tableau de taille 49, et qu'à ce moment-là une augmentation de la taille du tableau est nécessaire, tandis qu'à l'ajout du 100e élément, il restait de la place dans le tableau. Le prochain agrandissant dans une liste où la taille n'a pas été prédéfinie s'effectue à l'ajout du 105e élément et nécessite alors 548 opérations.

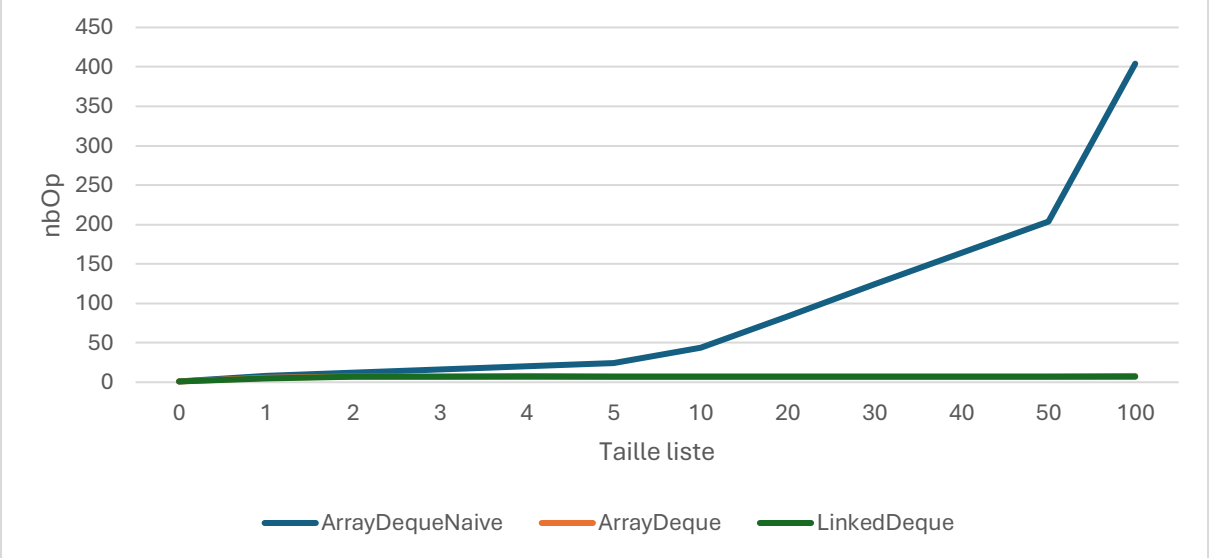
Un moyen d'éviter que cela arrive serait d'initialiser le tableau de base à une centaine de cases, ce qui demanderait l'ajout de nombreuses données avant de devoir étendre le tableau. Le problème de cette solution, c'est que ça gaspillerait énormément de mémoire, pour au final avoir la même complexité temporelle.

offerLast(E e)	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(1)



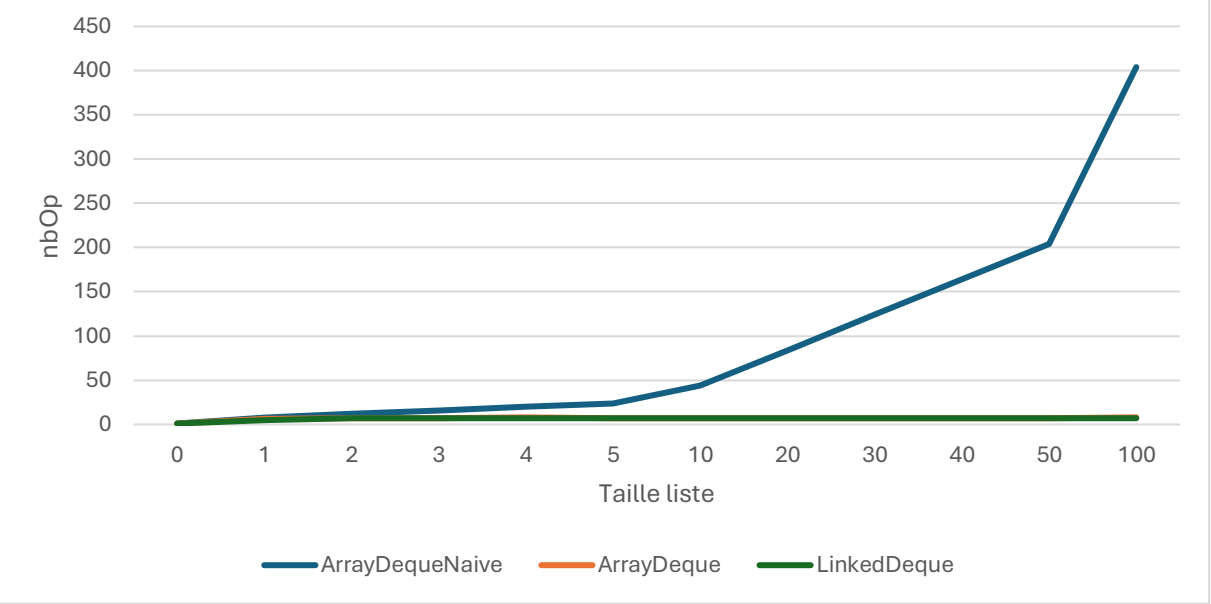
En comparant les deux graphiques, on peut voir que le nombre d'opérations nécessaire à l'ajout d'un élément au début ou à la fin d'une liste de taille similaire nécessite exactement le même nombre d'opérations.

removeFirst()	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(1)
LinkedDeQue	O(1)



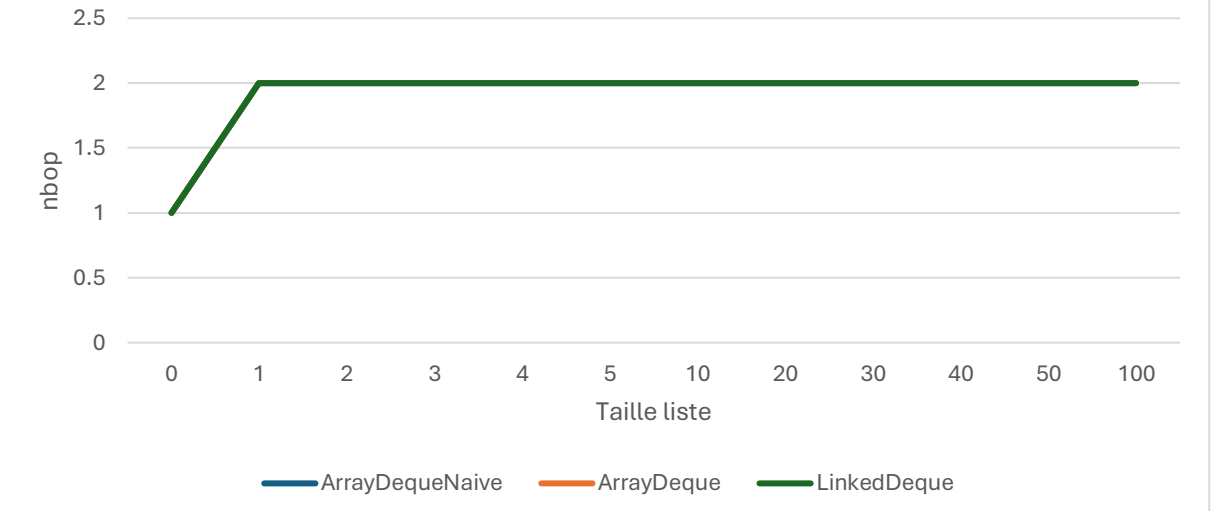
Supprimer un élément d'un tableau naïf demande beaucoup d'opérations, car il faut recréer un tableau de taille inférieure et copier les éléments à garder de l'ancien tableau à chaque fois.

removeLast()	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(1)
LinkedDeQue	O(1)



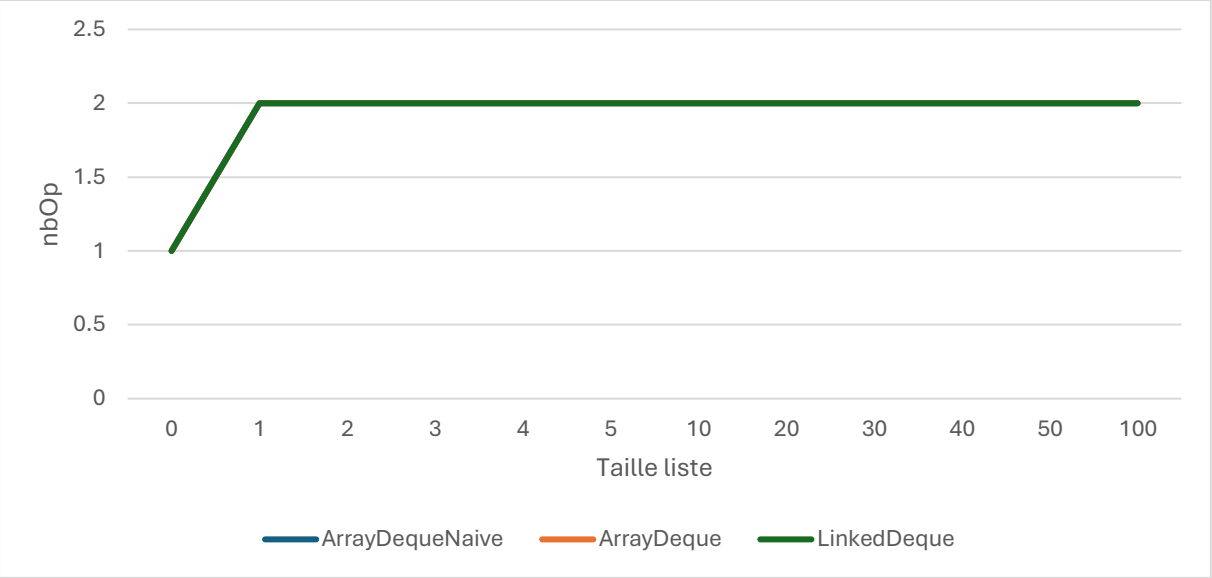
En comparant les deux graphiques, on peut voir que le nombre d'opérations nécessaire à la suppression d'un élément au début ou à la fin d'une liste de taille similaire nécessite exactement le même nombre d'opérations.

peekFirst()	
	mathématiquement
ArrayDequeNaive	O(1)
ArrayDeque	O(1)
LinkedDeQue	O(1)



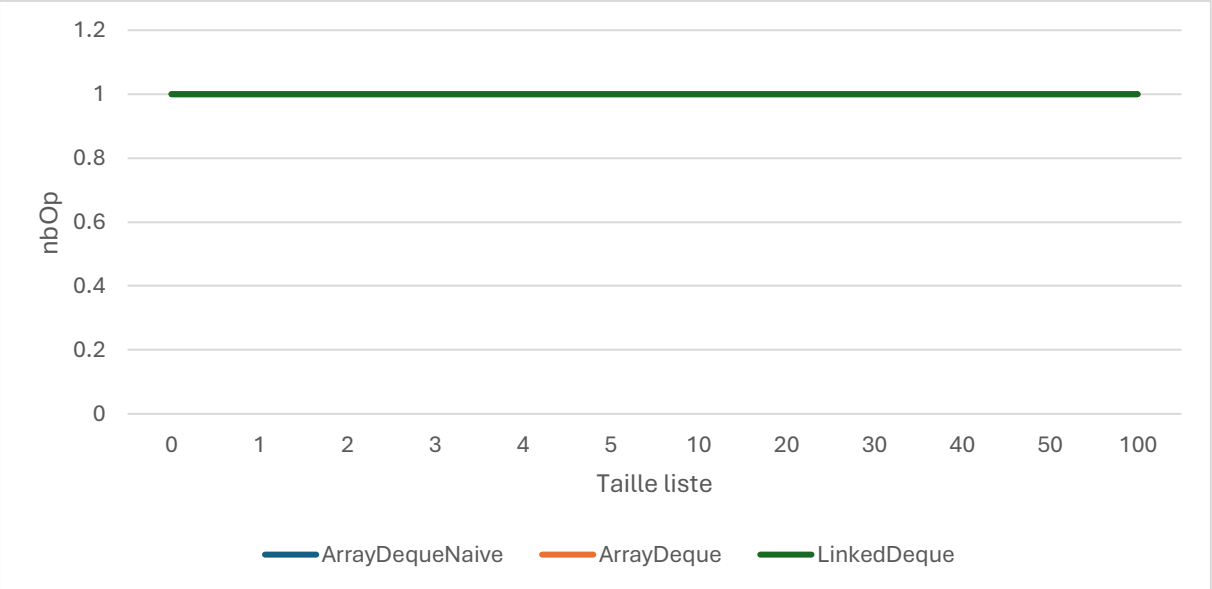
On peut constater que récupérer un élément au début de la liste, quel que soit le type de cette liste, ce fait de manière immédiate, grâce au fait que le programme sait toujours où se situe le premier élément.

peekLast()	
	mathématiquement
ArrayDequeNaive	O(1)
ArrayDeque	O(1)
LinkedDeQue	O(1)



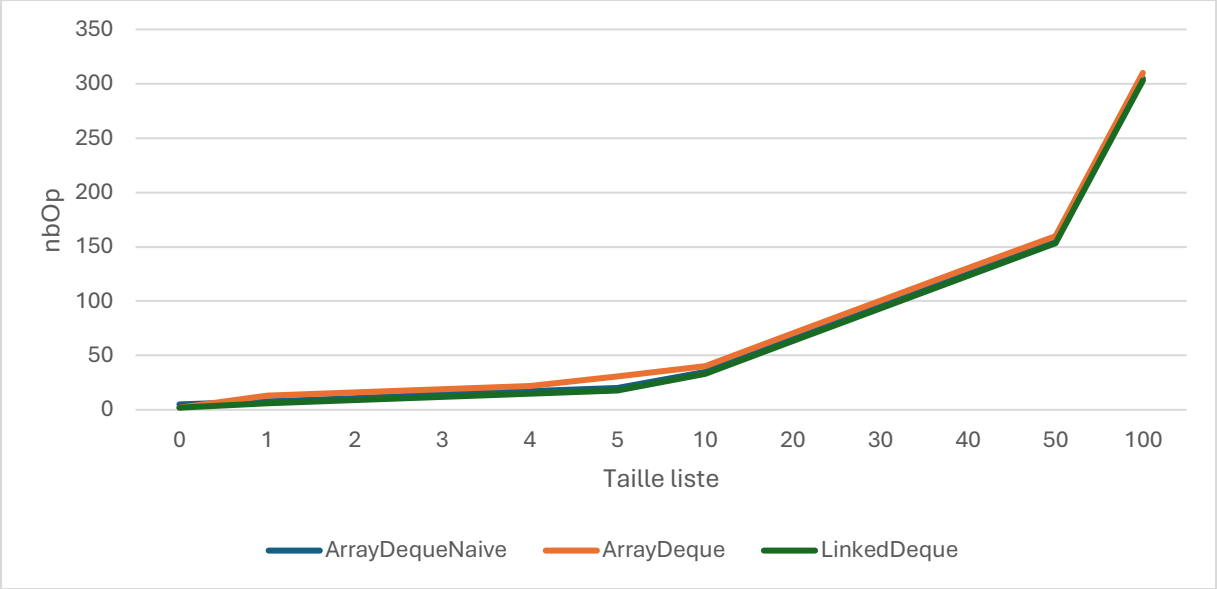
En comparant les deux graphiques, on peut voir que le nombre d'opérations nécessaire à la récupération d'un élément au début ou à la fin d'une liste de taille similaire nécessite exactement le même nombre d'opérations.

size()	
	mathématiquement
ArrayDequeNaive	O(1)
ArrayDeque	O(1)
LinkedDeQue	O(1)



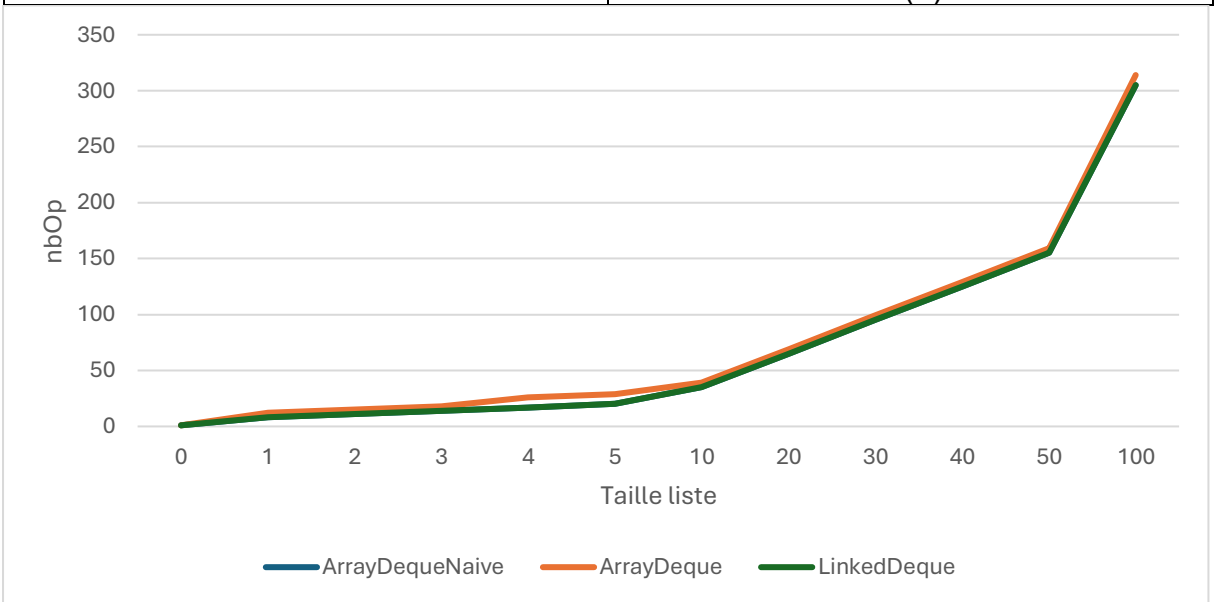
On peut constater que récupérer la taille de la liste, quel que soit le type de cette liste, ce fait de manière immédiate.

contains(Object o)	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(n)

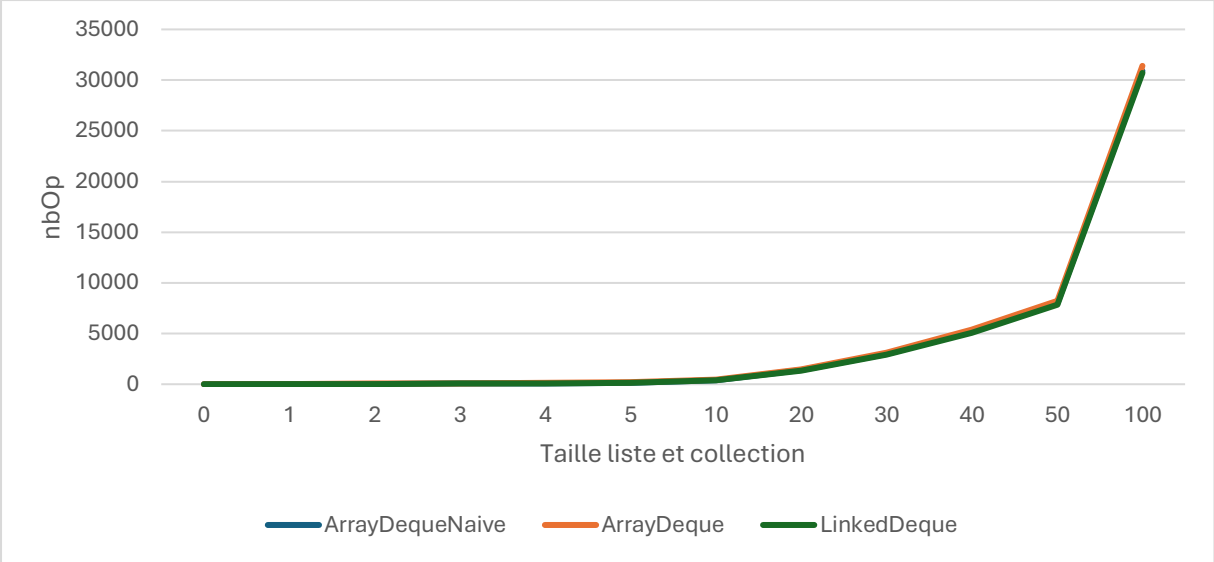


L'objet recherché n'était jamais présent dans la liste.

toString()	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(n)

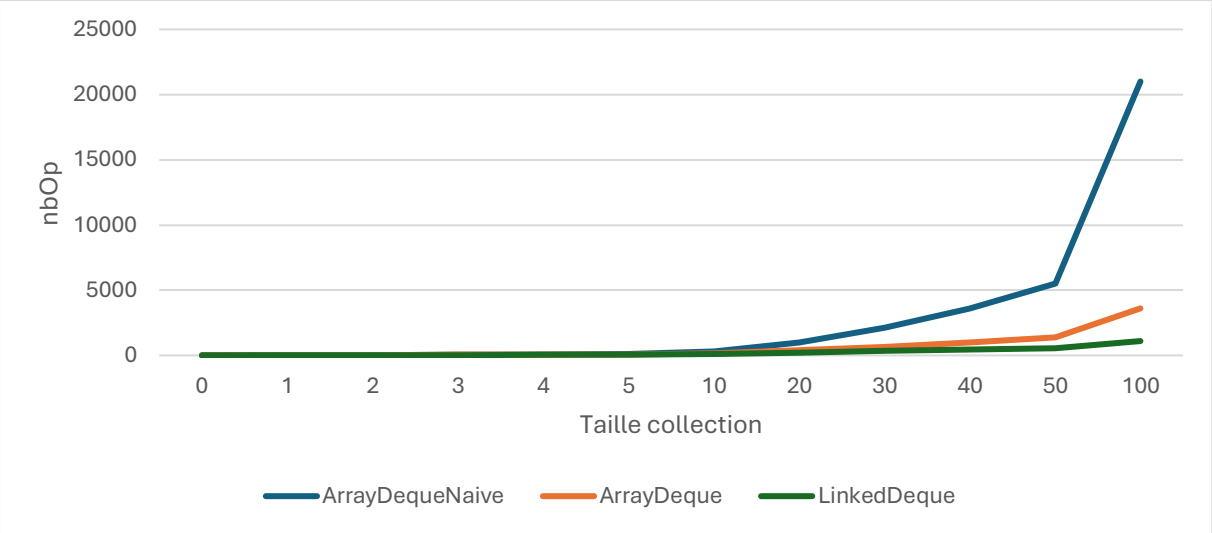


containsAll(Collection<?> c)	
	mathématiquement
ArrayDequeNaive	O(n * m)
ArrayDeque	O(n * m)
LinkedDeQue	O(n * m)



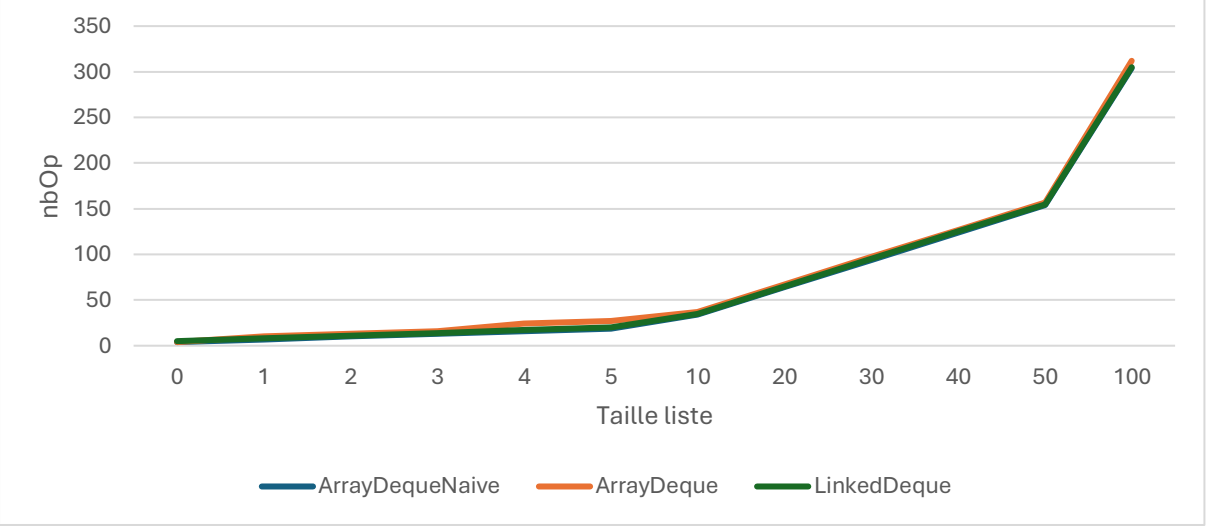
La taille de la collection utilisée était la même que la taille de la liste. Tous les objets présents dans la collection, exceptés le dernier, étaient les mêmes et présents au dernier emplacement de la liste. Le dernier objet de la collection était absent de la liste. Mathématiquement : (N = deque.length = array.length) ET (Pour tout i : 0 <= i < N – 1 : array[i] = deque[N – 1]) ET (Pour tout j : 0 <= j < N : array[N – 1] <> deque[j])

offerAll(Collection<? extends E> c)	
	mathématiquement
ArrayDequeNaive	O(n * m)
ArrayDeque	O(n * m)
LinkedDeQue	O(n)



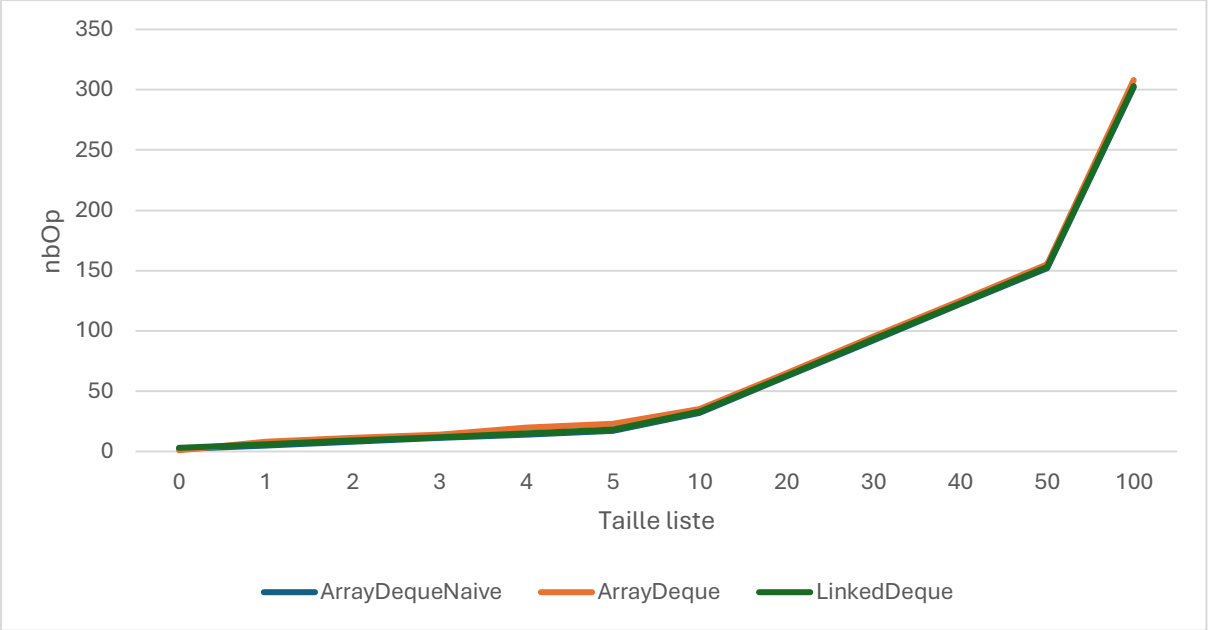
Lors de l'ajout, les listes étaient vides, seule la taille de la collection changeait.
La cause de la grande différence de courbe est la même que pour la méthode offerFirst : les trois listes doivent parcourir la collection pour pouvoir ajouter les éléments, d'où le O(n). Mais les deux listes utilisant un tableau, à l'inverse de la liste chaînée, ont un nombre de place limité, ce qui les oblige à créer de nouveaux tableaux plus grands de manière régulière, d'où le O(n * m). Et si ArrayDequeNaive a besoin d'autant d'opérations que cela, c'est parce que l'agrandissement ce fait à chaque nouvelle donnée.

containsIf(Predicate<? super E> filter)	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(n)

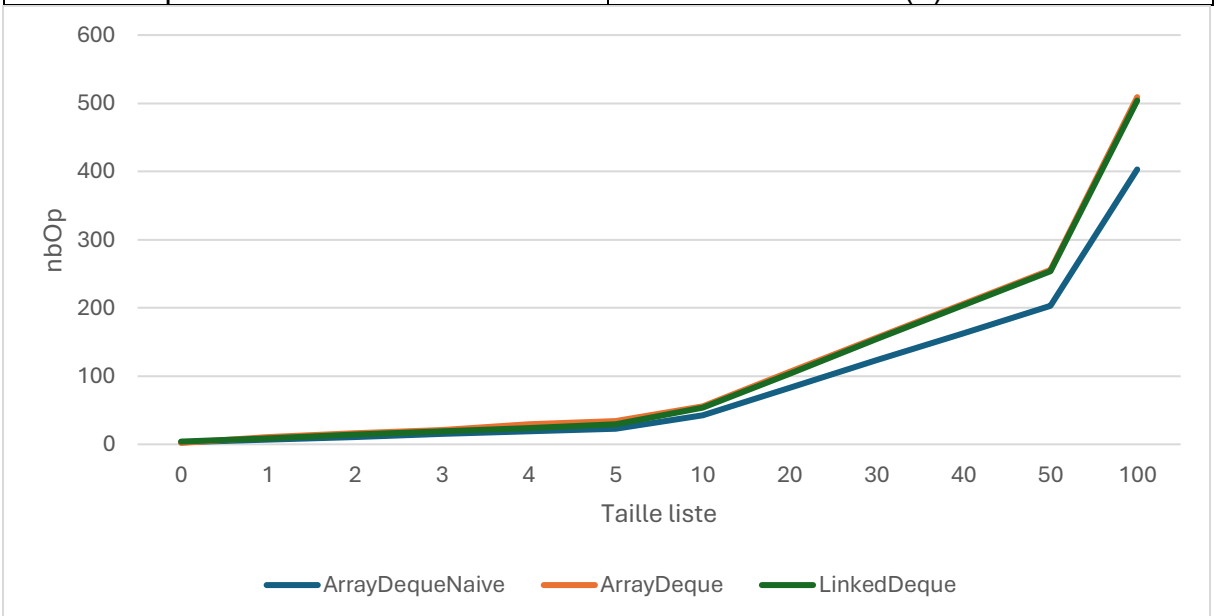


Le prédicat était faux pour tous les éléments de la liste.

forEach(Consumer<? super E> action)	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeQue	O(n)



toArray()	
	mathématiquement
ArrayDequeNaive	O(n)
ArrayDeque	O(n)
LinkedDeque	O(n)



ArrayDequeNaive pourrait avoir une complexité temporelle de O(1). Pour cela, il suffirait de renvoyer deque tel quel, mais alors tout le monde pourrait modifier les données contenues dans la liste.

On peut constater que la différence du temps d'exécution se fait ressentir lors d'ajout ou de retrait dans la liste. La méthode ArrayDequeNaive est très peu efficiente, tandis que les méthodes ArrayDeque et LinkedDeque se valent. Ce qui différencie ces deux dernières méthodes vient du fait que LinkedDeque n'ait jamais besoin de s'agrandir et requiert toujours le même petit nombre d'opérations, tandis que ArrayDeque a parfois besoin d'agrandir le tableau utilisé, ce qui nécessite alors un très grand nombre d'opérations.
De plus, lorsque l'on retire un élément, ArrayDeque n'est jamais rétrécie et utilise inutilement de l'espace mémoire, ce qui n'est pas le cas de ArrayDequeNaive et LinkedDeque.
Enfin, quand il s'agit de parcourir la liste pour retrouver un élément, les 3 types de liste sont équivalentes.

b. BigPriorityQueueInterace

offer(T e)	
	mathématiquement
ArrayBigPriorityQueueNaive	O(n)
ArrayBigPriorityQueue	O(n)
LinkedBigPriorityQueue	O(1)

poll()	
	mathématiquement
ArrayBigPriorityQueueNaive	O(n)
ArrayBigPriorityQueue	O(1)
LinkedBigPriorityQueue	O(1)

contains(Object o)	
	mathématiquement
ArrayBigPriorityQueueNaive	O(n)
ArrayBigPriorityQueue	O(n)
LinkedBigPriorityQueue	O(n)

peek()	
	mathématiquement
ArrayBigPriorityQueueNaive	O(1)
ArrayBigPriorityQueue	O(1)
LinkedBigPriorityQueue	O(1)

toString()	
	mathématiquement
ArrayBigPriorityQueueNaive	O(n)
ArrayBigPriorityQueue	O(n)
LinkedBigPriorityQueue	O(n)

En analysant ces résultats, mes conclusions sont les mêmes que pour le point précédent.

Conclusion

En conclusion, la liste ArrayDequeNaive, bien que tout aussi efficace lorsqu'il faut la parcourir, est à oublier.
La liste ArrayDeque n'est qu'une version améliorée de ArrayDequeNaive, plus efficiente, mais pas toujours optimale.
La liste LinkedDeque, quant à elle, est la plus flexible. Que ce soit pour ajouter, retirer ou parcourir une liste, elle allie efficacité temporelle et préservation de l'espace mémoire.

Ce travail m'a permis de bien comprendre les notions vues en cours, notamment la complexité temporelle avec laquelle j'avais plus de mal.

Annexes

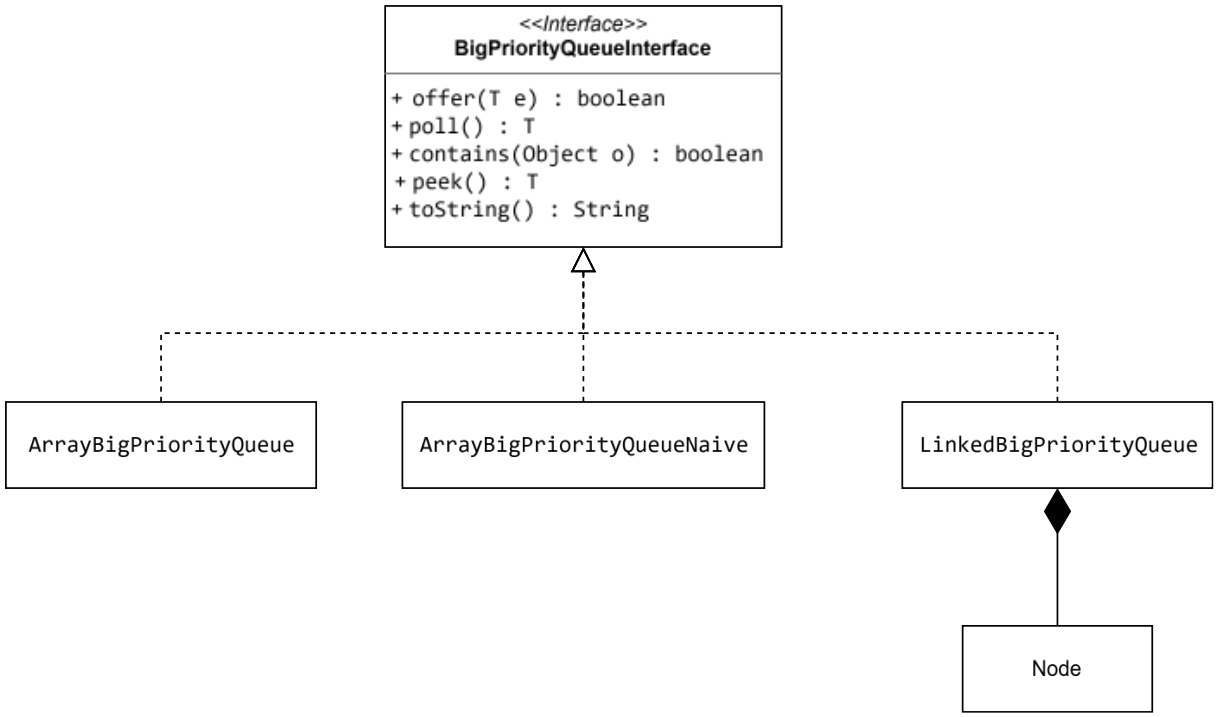


Fig 1 : Diagramme de classe de BigPriorityQueueInterface